

Lab 00 and Lab 00B: a plain-English study guide

Blur, sharpening, Gaussian kernels, convolution, interpolation and aliasing. Theory first, then the code, with worked numbers you can check by hand.

Every code block is preceded by pseudocode, so you know what to do and what to expect before you read any Python.

1. The big picture

Both labs are built around one idea:

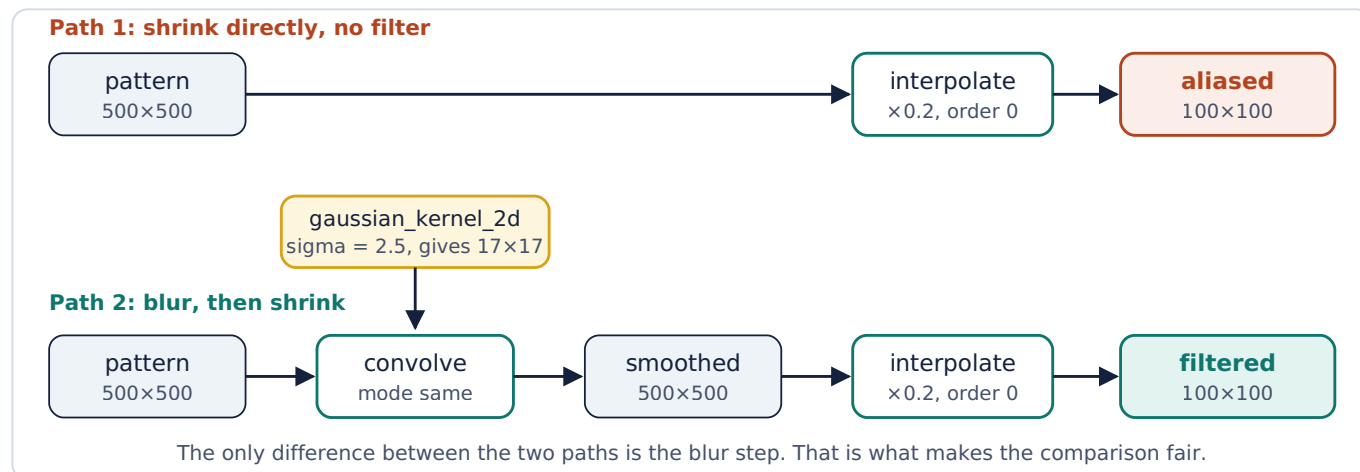
In plain words. An image contains detail at different "speeds": smooth areas that change slowly, and edges and fine texture that change quickly. A *filter* lets you keep some speeds and remove others. Shrinking an image safely needs a filter first.

What each lab is for

	Lab 00	Lab 00B
Class	<code>SimpleImageProcessing</code>	<code>ConvolutionInterpolator</code>
Methods you write	<code>add_blur</code> , <code>add_sharpen</code>	<code>gaussian_kernel_2d</code> , <code>convolve</code> , <code>interpolate</code> , plus <code>run_aliasing_demo</code> in the evaluation script
Main tools	<code>cv2.GaussianBlur</code> , <code>cv2.addWeighted</code> , <code>np.clip</code>	<code>np.outer</code> , <code>scipy.signal.convolve2d</code> , <code>skimage.transform.rescale</code>
Test images	Two photos (Sphinx, Sacré-Cœur)	A synthetic "chirp" pattern generated in code
Core idea	Blurring removes fine detail. Sharpening adds it back with extra weight.	Build the blur yourself, then see why an image must be blurred before it is shrunk
Output	<code>lab00_results.html</code>	<code>lab00B_results.html</code>

Lab 00 uses OpenCV as a black box: you call one function and get a blurred image. Lab 00B opens the box. You make the blur filter from its formula, apply it with a convolution, and then shrink the image with interpolation.

The Lab 00B pipeline in one picture



How to use this guide

- If a word confuses you, look it up in the glossary. It has one short definition per term.
- Sections 3, 4 and 5 each cover one method of Lab 00B: *theory*, *worked example*, *pseudocode*, *code*, *what the tests check*, in that order.
- Section 7 is the piece in the evaluation script that you have to complete.
- Section 11 is a one-page cheat sheet for the day you submit.

2. Foundations you need first

This section covers everything the two labs quietly assume you know. It also includes Lab 00 in detail (2.8), because Lab 00B builds directly on it.

2.1 An image is an array of numbers

A greyscale image is a grid of brightness values, one per pixel. In NumPy that grid is a 2D array called an `ndarray`. A colour image adds a third axis for the colour channels.

Kind of image	Shape	Meaning
Greyscale	<code>(H, W)</code>	H rows, W columns, one brightness per pixel
Colour	<code>(H, W, 3)</code>	The same, with 3 numbers per pixel (blue, green, red in OpenCV)

The lab docstrings write this as " $H \times W \times 3$ (or $H \times W$)" to say both cases are allowed.

Below, a tiny 3-row, 4-column greyscale image. You address a pixel as `image[row, col]`, with the row first. Rows count downward from the top.

Python

```
image = np.array([[12, 40, 90, 200],
                  [15, 45, 95, 210],
                  [20, 50, 100, 220]], dtype=np.uint8)

image.shape      # (3, 4)    3 rows, 4 columns
image[1, 2]      # 95       row 1, column 2
```

`dtype` is the number type stored in the array. Two matter here:

- `uint8` : whole numbers 0 to 255 ("unsigned 8-bit integer"). Standard for photos. Lab 00 uses it, and its tests check for it.
- `float64` : decimal numbers with no fixed range. The Lab 00B chirp pattern uses it, with values between -1 and 1 .

2.2 Why uint8 needs clipping (the wrap-around trap)

`uint8` cannot hold anything outside 0 to 255. When you force a number outside that range into it, NumPy does not cap it. It **wraps around**, like a car odometer.

Value before cast	After <code>.astype(np.uint8)</code>	What you wanted
300	44	255 (white)
325	69	255 (white)
-1	255	0 (black)
-18	238	0 (black)

A pixel that should be bright white would silently turn dark grey, or a black one turn nearly white. So the order is always:

1. compute in a wide type **2.** `np.clip(x, 0, 255)` **3.** `.astype(np.uint8)`

A detail worth knowing. OpenCV's `cv2.addWeighted` already caps ("saturates") results when its inputs are `uint8`. I checked: a true value of 325 comes out as 255. So in Lab 00 the `np.clip` is a safety net rather than the thing doing the work. Keep it anyway, because the lab asks for it, and because it matters the moment you compute in NumPy arithmetic instead.

In Lab 00B none of this applies. The chirp is float and the methods return floats, so there is no clip or cast.

2.3 Spatial frequency: how fast brightness changes

Spatial frequency is how quickly brightness changes as you move across an image. It is measured in **cycles per pixel**: one cycle is one light-to-dark-to-light repeat.

Frequency	Looks like	Examples
0 (constant)	Perfectly flat	A blank wall
Low	Slow, smooth changes	A clear sky, gentle shading on a cheek
High	Rapid changes	Sharp edges, text, grass, fabric weave
0.5 (the maximum)	Black, white, black, white on every pixel	A fine checkerboard

To convert: **period = 1 / frequency**. A frequency of 0.1 cycles/pixel repeats every 10 pixels. A frequency of 0.5 repeats every 2 pixels, which is the fastest a pixel grid can show.

In plain words. Low frequency means "the big picture". High frequency means "the fine detail". A blur removes fine detail. A sharpen boosts it. Everything in both labs is about which frequencies survive.

2.4 Filters and kernels

A **filter** changes an image's frequency content. The most common way to build one is a **kernel**: a small grid of numbers (weights). To filter, you slide the kernel over the image. At each position you multiply the kernel's weights by the pixels underneath and add them up. The result becomes the new value for the centre pixel.

Here is a 3×3 kernel that averages a pixel with its neighbours, giving the centre the most weight:

1/16	2/16	1/16
2/16	4/16	2/16
1/16	2/16	1/16

- A **low-pass** filter keeps slow changes and removes fast ones. This is a blur. A Gaussian is low-pass.
- A **high-pass** filter keeps fast changes and removes slow ones. That gives you edges.
- Kernel sizes are **odd** (3, 5, 15, 17). That way there is a true centre pixel for the result to be assigned to.
- A bigger kernel, or a bigger sigma, averages over a wider area. That means a stronger blur.

2.5 The Fourier transform, the spectrum and "energy"

Lab 00's tests measure frequencies with the **FFT** (Fast Fourier Transform). The idea is like hearing a musical chord and identifying the individual notes. The Fourier transform takes an image and reports how much of each spatial frequency it contains.

- The **magnitude spectrum** is a picture of those amounts. After `fftshift` the centre is the lowest frequency and the edges are the highest.
- **Energy** is the sum of the squared magnitudes over some region of that spectrum.
- **HF energy** is the energy in the high-frequency half. **LF energy** is the energy in the low-frequency half. The **HF-to-LF ratio** is the first divided by the second.

The tests then ask two simple questions:

Test	Expected	Why
Blur	HF energy goes down	A low-pass filter removes fast detail
Sharpen	HF/LF ratio goes up	Unsharp masking adds extra fast detail on top

I measured this on a standard 512×512 test photo, splitting "high" from "low" at 0.25 cycles/pixel from the centre. That split is my own, and the lab's test may draw the line elsewhere, but the direction is what matters:

	HF energy	HF/LF ratio
Original	4.4×10^{12}	0.0029
Blurred	3.3×10^{11} (down about 13×)	0.0002
Sharpened	2.3×10^{13}	0.0149 (up about 5×)

2.6 Python essentials used by the labs

Classes, instances and `self`

A **class** is a blueprint that bundles data and behaviour. Its functions are called **methods**. You **instantiate** the class to get an **instance** (an actual object), then call methods with dot notation.

The snippet below shows the pattern. `ci` is the instance:

Python

```
ci = ConvolutionInterpolator()          # create the instance
kernel = ci.gaussian_kernel_2d(2.5)    # call a method on it
```

`self` is the instance a method is being called on. It is the first parameter of every method, but you never type it in a call. Python passes it in for you. So `ci.convolve(image, kernel)` is the same as `ConvolutionInterpolator.convolve(ci, image, kernel)`.

Positional vs keyword arguments

- A **positional** argument is matched by its position: `ci.convolve(image, kernel)`. It must always be given.
- A **keyword** argument is matched by name: `mode="valid"`. Order does not matter, and they are usually optional because a default exists.

`**kwargs` and `kwargs.get`

`**kwargs` collects every keyword argument the caller passes into a dictionary named `kwargs`. Then `kwargs.get("mode", "same")` means: "look for `mode` in that dictionary; if it is missing, use `"same"`".

Call	What <code>kwargs</code> holds	Result of <code>kwargs.get("mode", "same")</code>
<code>ci.convolve(img, k)</code>	<code>{}</code>	<code>"same"</code> (default used)
<code>ci.convolve(img, k, mode="valid")</code>	<code>{"mode": "valid"}</code>	<code>"valid"</code>
<code>ci.convolve(img, k, mode="full", boundary="fill")</code>	<code>{"mode": "full", "boundary": "fill"}</code>	<code>"full"</code>

The benefit: the method signature stays `(self, image, **kwargs)` forever, and callers only name the settings they care about.

2.7 The NumPy toolbox for these labs

Function	What it does	Example
<code>np.arange(a, b)</code>	Whole numbers from <code>a</code> up to <i>but not including</i> <code>b</code>	<code>np.arange(-3, 4)</code> gives <code>[-3 -2 -1 0 1 2 3]</code>
<code>np.ceil(x)</code>	Rounds <i>up</i> . Returns a float, so wrap it in <code>int()</code>	<code>np.ceil(7.5)</code> gives <code>8.0</code>
<code>np.exp(x)</code> , <code>np.sqrt(x)</code> , <code>np.pi</code>	e^x , square root, the constant π	
<code>x ** 2</code> , <code>-(x ** 2)</code>	Arithmetic on a whole array acts on every element at once ("vectorised")	<code>np.arange(-2,3) ** 2</code> gives <code>[4 1 0 1 4]</code>
<code>np.outer(a, b)</code>	Grid whose entry <code>[i, j]</code> is <code>a[i] * b[j]</code>	Section 3.5
<code>arr.sum()</code>	Adds up every element	
<code>np.clip(x, lo, hi)</code>	Caps every value into <code>[lo, hi]</code>	<code>np.clip([-20, 100, 300], 0, 255)</code> gives <code>[0 100 255]</code>
<code>arr.astype(t)</code>	Converts to another dtype	<code>x.astype(np.uint8)</code>

2.8 Lab 00 in detail (blur and sharpen)

Gaussian blur

A Gaussian blur replaces each pixel with a weighted average of its neighbours, where nearby pixels count most and far ones count little. It is a low-pass filter. Section 3 explains where the weights come from.

`cv2.GaussianBlur(image, (ksize, ksize), 0)` takes three things:

- the image;
- the kernel size as a *tuple* (width, height). They are the same number here, but OpenCV asks for both;
- a sigma. Passing `0` tells OpenCV to work sigma out from the kernel size.

Notice the direction. In Lab 00 you choose the kernel size and OpenCV derives sigma. In Lab 00B you choose sigma and *you* derive the kernel size (Section 3.3).

The kernel size must be odd, which is why the solution adds 1 when it is even.

The unsharp mask

The name comes from a darkroom trick, and the computation is short:

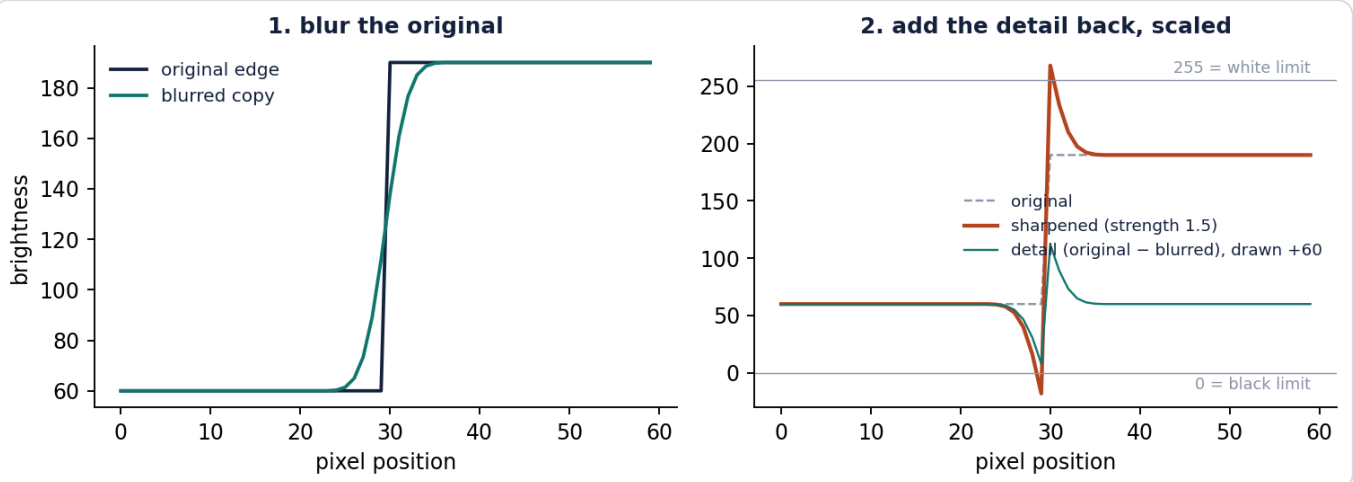
$$\text{sharpened} = \text{original} + \text{strength} \times (\text{original} - \text{blurred}) = (1 + \text{strength}) \times \text{original} - \text{strength} \times \text{blurred}$$

1. Blur the image. The blurred copy has lost its fine detail.
2. $\text{original} - \text{blurred}$ is therefore **only the fine detail** (edges, texture).
3. Add that detail back on top of the original, multiplied by strength . Edges get exaggerated, so the picture looks sharper.

Worked example, one pixel. Original = 100, blurred = 80, strength = 1.5.

Detail = $100 - 80 = 20$. Sharpened = $100 + 1.5 \times 20 = 130$. Same result from the second form: $2.5 \times 100 - 1.5 \times 80 = 250 - 120 = 130$.

In a flat area, original = blurred, so detail = 0 and nothing changes. Sharpening only touches edges. Near a very bright edge you can get $250 + 1.5 \times 50 = 325$, which is over 255. That is where clipping comes in.



A step edge in one dimension. The blurred copy rounds the edge. Adding the scaled difference makes the sharpened line overshoot on both sides of the edge, which is what makes an edge look crisper. The overshoot goes past 0 and 255, so it must be clipped.

The two methods: pseudocode, then code

Pseudocode: read this first

```
add_blur(image, **kwargs):
    1. ksize = kwargs "ksize", default 15
    2. if ksize is even, add 1 (kernel size must be odd)
    3. blurred = GaussianBlur(image, kernel (ksize, ksize), sigma 0)
    4. return blurred          # already uint8, no clip or cast needed

add_sharpen(image, **kwargs):
    1. ksize = kwargs "ksize", default 15
       strength = kwargs "strength", default 1.5
    2. blurred = add_blur(image, ksize)
    3. sharpened = addWeighted(image, 1 + strength, blurred, -strength, 0)
    4. sharpened = clip(sharpened, 0, 255)      # BEFORE the cast
    5. sharpened = sharpened as uint8
    6. return sharpened
```

Python

```
def add_blur(self, image, **kwargs):
    ksize = kwargs.get("ksize", 15)
    if ksize % 2 == 0:                # kernel size has to be odd
        ksize += 1
    blurred = cv2.GaussianBlur(image, (ksize, ksize), 0)
    return blurred

def add_sharpen(self, image, **kwargs):
    ksize = kwargs.get("ksize", 15)
    strength = kwargs.get("strength", 1.5)

    blurred = self.add_blur(image, ksize=ksize)
    sharpened = cv2.addWeighted(image, 1 + strength, blurred, -strength, 0)

    sharpened = np.clip(sharpened, 0, 255)    # clip before the cast
    sharpened = sharpened.astype(np.uint8)
    return sharpened
```

`cv2.addWeighted(src1, alpha, src2, beta, gamma)` computes $\alpha \cdot \text{src1} + \beta \cdot \text{src2} + \gamma$. Here $\alpha = 1 + \text{strength}$, $\beta = -\text{strength}$, $\gamma = 0$, which is exactly the formula above.



What the Lab 00 evaluation panels look like, shown on a standard test photo with the default settings. Blurred loses fine detail. Sharpened has crisper edges and slightly bright halos beside strong edges.

What the Lab 00 pytests check

- Blur lowers high-frequency FFT energy (both images).
- Sharpen raises the HF-to-LF energy ratio (both images).
- Output shape equals input shape.
- Output dtype is `uint8`.

3. The Gaussian kernel

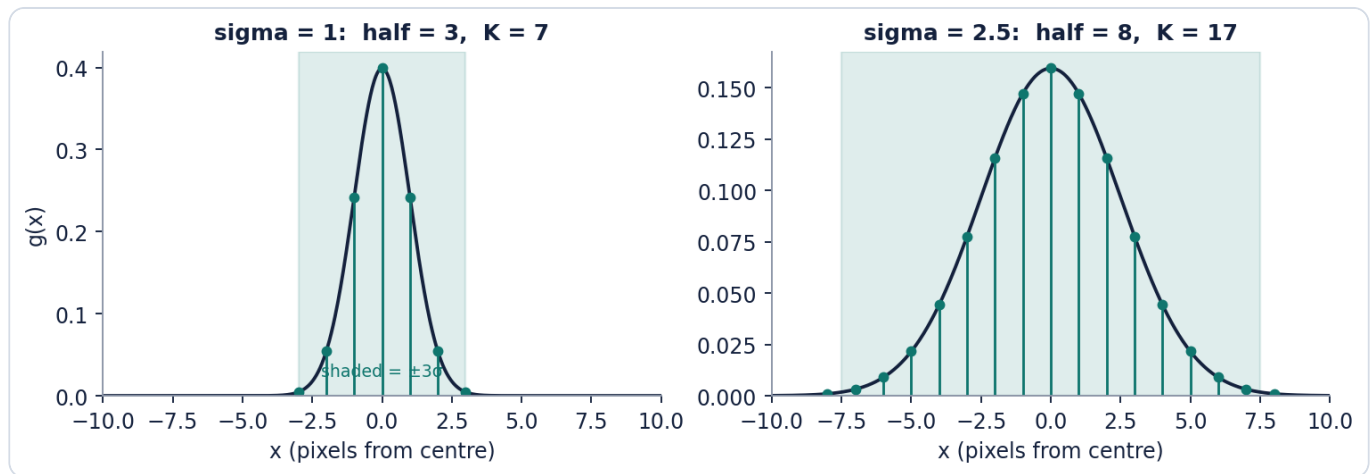
This is the first method of Lab 00B: `gaussian_kernel_2d(sigma)`. Input: one number, sigma. Output: a square grid of weights that add up to 1.

3.1 The bell curve and sigma

The **Gaussian** (or normal) curve is the familiar bell shape. It is highest in the middle and falls smoothly to nearly zero on both sides. As a blur weight, it means: "the centre pixel counts most, near neighbours count a lot, far neighbours count almost nothing".

Sigma (σ) is the standard deviation. It controls the width of the bell.

- Small σ gives a narrow spike, so only the immediate neighbours matter. A light blur.
- Large σ gives a wide hill, so pixels from far away contribute. A heavy blur.



The smooth curve is the mathematical Gaussian. The dots are the values you actually store in the kernel: the curve sampled at whole-number positions. The shaded band is $\pm 3\sigma$.

3.2 The formula, piece by piece

$$g(x) = 1 / (\sigma \cdot \sqrt{2\pi}) \cdot \exp(-x^2 / (2\sigma^2))$$

Part	What it does
<code>x</code>	Distance from the kernel's centre, in pixels. The centre is 0.
<code>x²</code>	Makes the curve symmetric (-2 and +2 give the same value) and makes it fall off faster the further you go.
<code>2σ²</code>	Sets how fast the fall-off is. Bigger σ means a slower fall, so a wider bell.
<code>exp(-...)</code>	Turns "bigger distance" into "smaller weight". At $x = 0$ it gives exactly 1. It never quite reaches 0.
<code>1 / (σ · √(2π))</code>	A constant that makes the area under the continuous curve equal 1. You will normalise the kernel yourself at the end (3.6), so this constant cancels out numerically. The lab still wants it in your code because the test builds its reference from the formula.

3.3 From a curve to a finite grid: the $\pm 3\sigma$ rule

A kernel is a finite grid, so you cannot keep the whole curve. You sample it at whole-number positions `x = -half, ..., -1, 0, 1, ..., half`.

How large should `half` be? About **99.7% of a Gaussian's weight lies within $\pm 3\sigma$ of its centre**. Beyond that the tails are tiny, so you can stop there:

$$\text{half} = \text{ceil}(3 \times \sigma) \quad K = 2 \times \text{half} + 1$$

- `ceil` rounds *up*, so the kernel always covers the full $\pm 3\sigma$ span.

- K is always odd, so there is a genuine centre pixel.
- Number of samples = `half` on the left + 1 centre + `half` on the right = $2 \cdot \text{half} + 1$.

σ	3σ	<code>half = ceil(3σ)</code>	$K = 2 \cdot \text{half} + 1$
0.5	1.5	2	5
1	3	3	7
1.5	4.5	5	11
2	6	6	13
2.5 (the lab's value)	7.5	8	17
3	9	9	19
5	15	15	31

Watch out: `ceil`, not `int`. `int(3 * 2.5)` gives 7 (it drops the .5), so K would be 15 instead of 17. The kernel would then stop short of $\pm 3\sigma$, and the pytest that checks the size would fail. Use `int(np.ceil(3 * sigma))`.

3.4 Worked example: $\sigma = 1$

Here `half = 3`, so you sample at $x = -3 \dots 3$ (7 numbers). The constant $1/(\sigma\sqrt{2\pi})$ is 0.3989 when $\sigma = 1$.

x	-3	-2	-1	0	1	2	3
$\exp(-x^2/2)$	0.0111	0.1353	0.6065	1	0.6065	0.1353	0.0111
$g(x) = 0.3989 \times \text{that}$	0.0044	0.0540	0.2420	0.3989	0.2420	0.0540	0.0044

This is the 1D vector `g`. It is symmetric, peaks at the centre, and is almost 0 at the ends.

3.5 The outer product: from 1D to 2D

To make a 2D kernel, multiply the 1D Gaussian in the x-direction by the 1D Gaussian in the y-direction.

`np.outer(g, g)` does this: entry `[i, j] = g[i] * g[j]`.

A tiny example with `a = [1, 2, 1]`:

1×1	1×2	1×1
2×1	2×2	2×1
1×1	1×2	1×1

1	2	1
2	4	2
1	2	1

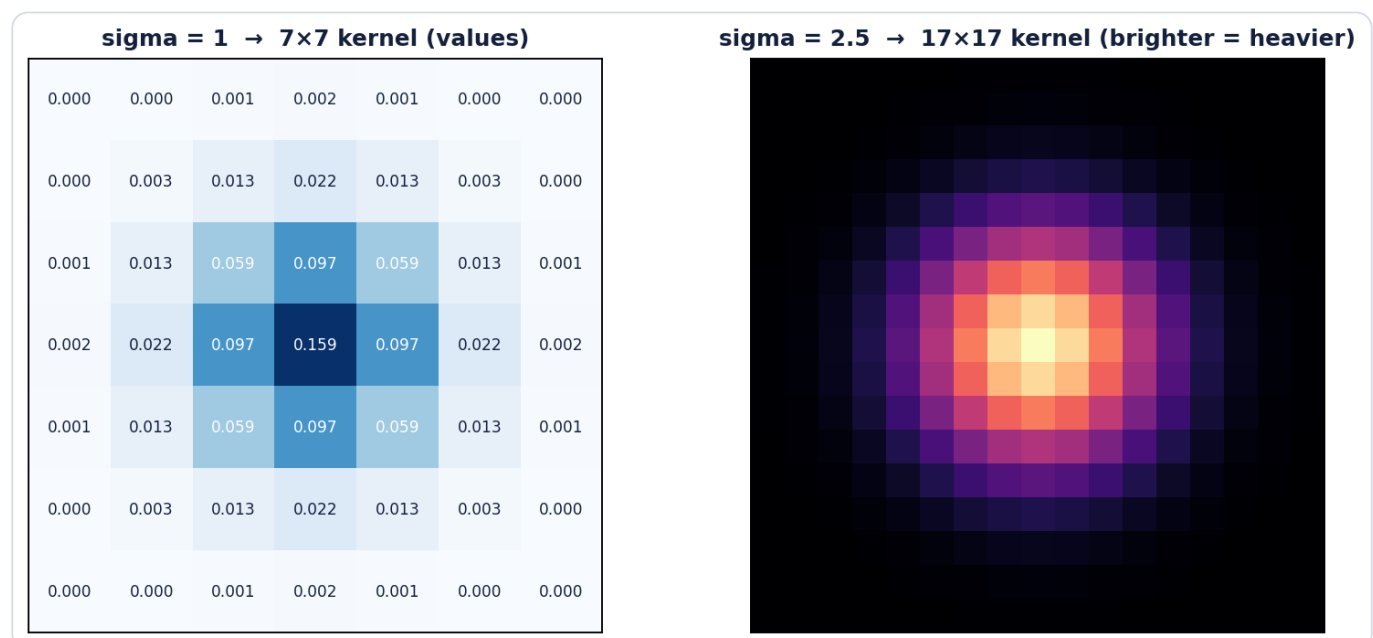
the rule: row value × column value

`np.outer([1, 2, 1], [1, 2, 1])`

Divide that by its sum (16) and you get exactly the 3×3 blur kernel from Section 2.4. Your real kernel is the same thing with Gaussian numbers instead of 1, 2, 1.

Why it is legitimate: the 2D Gaussian $\exp(-(x^2 + y^2)/2\sigma^2)$ equals $\exp(-x^2/2\sigma^2) \times \exp(-y^2/2\sigma^2)$. A 2D Gaussian really is a product of two 1D Gaussians. This property is called **separability**.

Do not confuse `np.outer(g, g)` with `g * g`. The second multiplies element by element and gives another 1D vector.



Left: the $\sigma = 1$ kernel with its actual values. The centre weight is about 0.159 and the corners are almost 0. Right: the $\sigma = 2.5$ kernel the lab uses, 17×17, drawn as brightness.

3.6 Normalisation: why the weights must sum to 1

You finish by dividing the kernel by its own sum: `kernel = kernel / kernel.sum()`.

- **Brightness preservation.** Suppose a flat region has value c everywhere. The filter gives $\sum(\text{weight} \times c) = c \times \sum(\text{weights})$. If the weights sum to 1, the result is c and the region stays the same brightness. If they summed to 0.9 the image would darken. If they summed to 1.3 it would brighten.
- **It really is needed.** The $\sigma = 1$ kernel sums to 0.99946 before normalising, not exactly 1, because you cut off the tails and sampled discretely.
- **The constant cancels.** If every entry is multiplied by some constant c , the sum is also multiplied by c , and dividing by the sum removes it. So the normalised kernel is the same whether or not you included $1/(\sigma\sqrt{2\pi})$.

- After normalising, the sum is 1 up to rounding, for example 0.9999999999999998. That is why tests compare with a tolerance (`allclose`) rather than `==` .

3.7 The method: pseudocode, then code

Pseudocode: read this first

```
gaussian_kernel_2d(sigma):
    1. half = ceil(3 * sigma), converted to int
    2. x     = the whole numbers from -half to +half (2*half + 1 of them)
    3. g     = 1/(sigma*sqrt(2*pi)) * exp( -x**2 / (2*sigma**2) )
    4. kernel = outer(g, g)           # every entry [i, j] = g[i] * g[j]
    5. kernel = kernel / sum(kernel) # make the weights add up to 1
    6. return kernel
```

Expect for sigma = 2.5: shape (17, 17), float dtype, sum = 1,
largest value at the centre, index (8, 8)

Python

```
def gaussian_kernel_2d(self, sigma):
    half = int(np.ceil(3 * sigma))
    x = np.arange(-half, half + 1)           # +1 because arange excludes the end

    g = 1.0 / (sigma * np.sqrt(2 * np.pi)) * np.exp(-(x ** 2) / (2 * sigma ** 2))

    kernel = np.outer(g, g)
    kernel = kernel / kernel.sum()
    return kernel
```

Why half + 1 ? `np.arange(-8, 8)` stops at 7 (the end is excluded), giving 16 numbers.
`np.arange(-8, 9)` gives `-8 ... 8`, which is the 17 you need.

What the tests check

- Returns a $K \times K$ float array whose size follows the $\pm 3\sigma$ rule.
- Sums to 1.
- Values match a reference built from the 1D formula and `np.outer` .
- The peak is at the centre.

4. Convolution

The second method: `convolve(image, kernel, **kwargs)` . It applies the kernel from Section 3 to an image. This one is short to code, because you hand the work to `scipy.signal.convolve2d` . The real learning is in understanding what it does and what its settings mean.

4.1 What convolution does

Convolution means: slide the kernel across the image, and at every position multiply the overlapping numbers and add them up.

Worked example in one dimension. Signal $[0, 0, 10, 0, 0]$, kernel $[0.25, 0.5, 0.25]$.

At the centre pixel: $0.25 \times 0 + 0.5 \times 10 + 0.25 \times 0 = 5$. At the pixel to its left: $0.25 \times 0 + 0.5 \times 0 + 0.25 \times 10 = 2.5$. Result: $[0, 2.5, 5, 2.5, 0]$.

The single bright spike has been spread out into a soft bump. That is blur.

The same in two dimensions

Take a 5×5 image of 10s with one bright pixel of 90 in the middle, and the 3×3 kernel from Section 2.4.

10	10	10	10	10
10	10	10	10	10
10	10	90	10	10
10	10	10	10	10
10	10	10	10	10

image

10	10	10	10	10
10	15	20	15	10
10	20	30	20	10
10	15	20	15	10
10	10	10	10	10

after convolution (mode "same")

Check the centre by hand: the 90 has weight $4/16 = 22.5$, and the eight 10s have total weight $12/16$, adding 7.5. Total: 30. The bright dot has been smeared into its neighbours.

4.2 Convolution vs correlation (the "flip")

True convolution **flips the kernel** before sliding it. *Correlation* slides it without flipping. With an asymmetric kernel $[1, 2, 3]$ and a single spike:

Spike input	Convolution result	Correlation result
[0, 0, 1, 0, 0]	[0, 1, 2, 3, 0]	[0, 3, 2, 1, 0]

A Gaussian kernel is **symmetric** (the same forwards and backwards), so flipping changes nothing. For this lab the two are numerically identical. The distinction matters in later labs with asymmetric kernels such as edge detectors.

4.3 mode : how big is the output?

When the kernel hangs over the edge of the image, you have to decide what to do. **mode** chooses the output size. For an $H \times W$ image and $K \times K$ kernel:

mode	Meaning	Output size	500×500 image, K = 17	5×5 image, K = 3
"same" (default here)	Pad the edges so the output matches the input	$H \times W$	500×500	5×5
"valid"	Only positions where the kernel fully fits. No padding.	$(H - K + 1) \times (W - K + 1)$	484×484	3×3
"full"	Every position with any overlap at all	$(H + K - 1) \times (W + K - 1)$	516×516	7×7

Use **"same"** when you want to compare the result against the original image, which is what you do in this lab.

For the 5×5 example above, **"valid"** gives just the inner 3×3 block:

15	20	15
20	30	20
15	20	15

4.4 boundary and fillvalue : what is beyond the edge?

At the border, the kernel covers pixels that do not exist. **boundary** chooses what to pretend is there. It only matters when padding is being done, i.e. **"same"** or **"full"**. With **"valid"** it has no effect.

boundary	Pretends the missing pixels are...	Effect
"symm" (default here)	A mirror image of the pixels near the edge	Blends naturally with the image. No artificial border.
"fill"	A constant, <code>fillvalue</code> (default 0)	Zeros drag the average down, so edges darken.
"wrap"	The opposite side of the image (as if tiled)	Edge pixels get contaminated by the far edge.

To see the difference, take a 5×5 image that is uniformly 100. Its corner pixel *should* stay 100 after blurring.

boundary	Corner value after blur	Verdict
"symm"	100	Correct
"fill" (with 0)	56.25	Much too dark. Only 9/16 of the kernel weight lands on real pixels.

Or take a 5×5 image that counts up by 4 row by row (0, 4, 8, ... 96), so its top-left corner is 0: with "symm" it comes out as 6, with "fill" it is 4.5, and with "wrap" it is 30, dragged up by values from the far side of the image. This is why "symm" is the default.

`fillvalue` is just the constant used by `boundary="fill"`. It is ignored otherwise.

4.5 The method: pseudocode, then code

Pseudocode: read this first

```
convolve(image, kernel, **kwargs):
    1. mode      = kwargs "mode",      default "same"
       boundary = kwargs "boundary",  default "symm"
       fillvalue = kwargs "fillvalue", default 0
    2. result = convolve2d(image, kernel, mode, boundary, fillvalue)
    3. return result          # float in, float out: no clip or cast
```

Expect (500×500 image, 17×17 kernel):

default -> (500, 500) mode="valid" -> (484, 484) mode="full" -> (516, 516)

Python

```
def convolve(self, image, kernel, **kwargs):
    mode = kwargs.get("mode", "same")
    boundary = kwargs.get("boundary", "symm")
    fillvalue = kwargs.get("fillvalue", 0)

    result = convolve2d(image, kernel, mode=mode,
                        boundary=boundary, fillvalue=fillvalue)
    return result
```

What the tests check

- Matches `scipy.signal.convolve2d` called directly with the same arguments.
- Preserves the input shape by default.
- Shrinks correctly with `mode="valid"`.
- Respects the `boundary` setting.
- Leaves a flat (constant) region unchanged, which only works because the kernel sums to 1.

5. Interpolation

The third method: `interpolate(image, **kwargs)`. It resamples an image to a new size.

5.1 What interpolation is

To change an image's size you create a new pixel grid, and for every new pixel you must **estimate a value** from the old pixels. That estimate is **interpolation**.

- `scale_factor` is the new size relative to the old one. Below 1 the image shrinks (**downsampling**). Above 1 it grows (**upsampling**).
- The new shape is roughly `(H × scale, W × scale)`. For a 500×500 image: 0.2 gives 100×100, 0.5 gives 250×250, 2 gives 1000×1000.

5.2 `order` : how the estimate is made

order	Name	How each new pixel is estimated
0 (default here)	Nearest neighbour	Copy the value of the single closest old pixel. Nothing is averaged.
1	Bilinear	A weighted average of the 4 nearest old pixels
3	Bicubic	A smoother curve fitted through 16 nearby pixels

A worked example: shrink this 4×4 image by 0.5 (I ran both):

0	10	20	30
40	50	60	70
80	90	100	110
120	130	140	150

input (highlighted: the only 4 pixels order 0 uses)

50	70
130	150

order 0 (nearest)

25	45
105	125

order 1 (bilinear)

Order 0 simply **throws away 12 of the 16 pixels** and copies four. It has no idea what was in the pixels it skipped. That is exactly why it is the right choice for this lab: it applies zero smoothing of its own, so whatever appears in the result is caused only by whether you filtered beforehand.

Upsampling a two-pixel row `[0, 10]` by 2 gives `[0, 0, 10, 10]` with order 0 and `[2.5, 2.5, 7.5, 7.5]` with order 1.

5.3 `mode="reflect"`

Near the image border, the estimate can need pixels that do not exist. `mode="reflect"` mirrors the image at the edge to supply them. This is a separate setting from `convolve`'s `boundary`, and you always pass `'reflect'` here.

5.4 `anti_aliasing=False` : switch off the hidden filter

By default, `skimage.transform.rescale` secretly applies its own Gaussian blur when it detects that you are shrinking. Its rule is $\sigma = (\text{shrink factor} - 1) / 2$, which for `×0.2` (a factor of 5) is $\sigma = 2$. I confirmed this in the library source.

If you left it on, your "unfiltered" path would already be filtered, and the lab's whole comparison would disappear. So `anti_aliasing=False` is **not optional**. The test even checks that your output does *not* match what `rescale` gives with `anti_aliasing=True`.

5.5 The method: pseudocode, then code

Pseudocode: read this first

```
interpolate(image, **kwargs):
    1. scale_factor = kwargs "scale_factor", default 0.5
       order       = kwargs "order",          default 0
    2. resampled = rescale(image, scale_factor,
                           order=order,
                           anti_aliasing=False,    # MUST be off
                           mode='reflect')

    3. return resampled
```

Expect: a 500x500 image with `scale_factor=0.2` gives shape (100, 100)

Python

```
def interpolate(self, image, **kwargs):
    scale_factor = kwargs.get("scale_factor", 0.5)
    order = kwargs.get("order", 0)

    resampled = rescale(image, scale_factor, order=order,
                        anti_aliasing=False, mode="reflect")
    return resampled
```

What the tests check

- Output shape matches `scale_factor`.

- Values match `rescale(..., anti_aliasing=False)` called directly.
- Values do **not** match `rescale(..., anti_aliasing=True)`, proving you add no smoothing of your own.

6. Aliasing: the theory

This is the "why" of the whole lab.

6.1 Sampling

A digital image is a set of **samples** of a real scene: each pixel records the brightness at one spot. Shrinking an image by 0.2 means keeping samples that are five original pixels apart. In effect, you resample the picture at a coarser spacing.

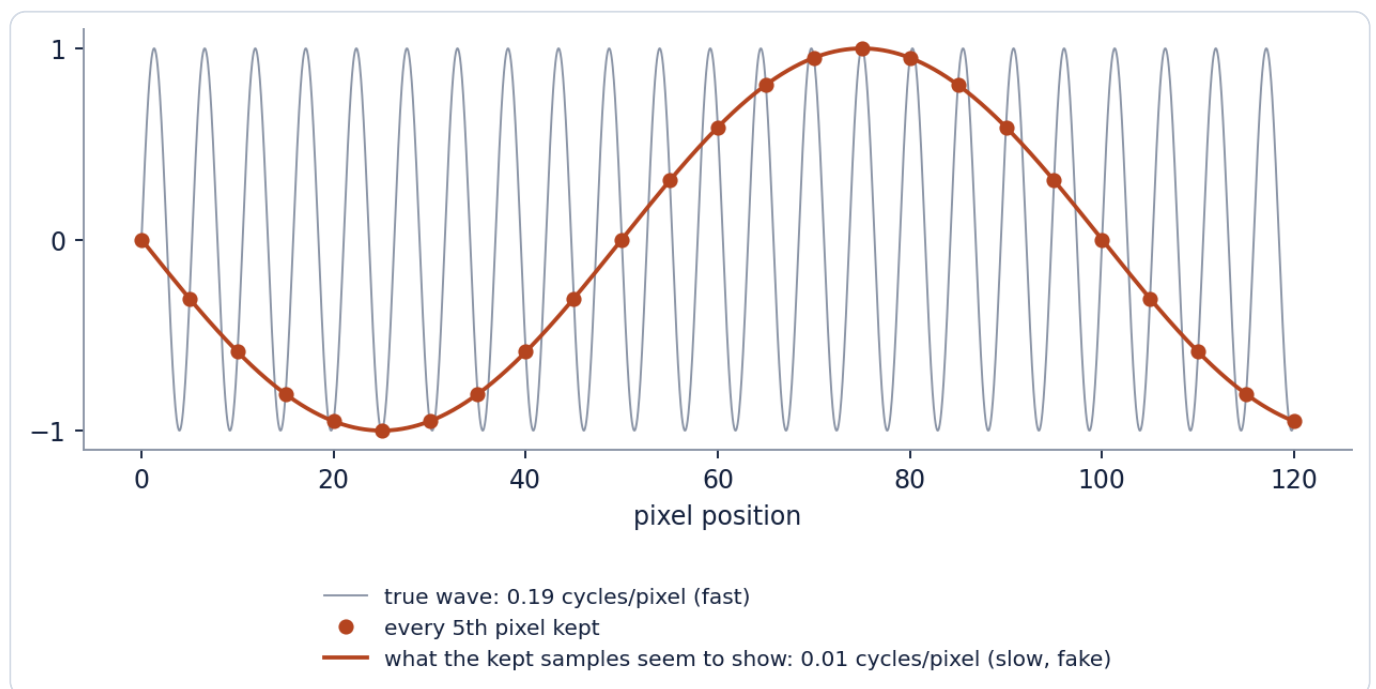
6.2 The Nyquist limit

To show a wave, you need at least **two samples per cycle**: one for the peak and one for the trough. So a grid can represent frequencies only up to **0.5 cycles per pixel**. This ceiling is the **Nyquist limit**.

Shrink by 0.2 and the new samples are 5 original pixels apart, so the limit falls to $0.5 / 5 = 0.1$ **cycles per original pixel**. Any detail faster than that cannot be represented in the 100×100 result.

6.3 What aliasing is

Detail above the limit does not just disappear. When you sample too sparsely, fast waves **masquerade as slow ones**. That false pattern is **aliasing**.



The grey wave is real and fast (0.19 cycles/pixel). Keeping only every 5th pixel (dots) gives numbers that also fit a slow wave (orange). The slow wave never existed in the original.

The numbers: between two kept samples the fast wave advances $0.19 \times 5 = 0.95$ of a cycle, which is almost a full turn. It looks as though it went *backwards* by 0.05 of a cycle per step. The apparent frequency is $|0.19 - 0.2| = 0.01$ cycles/pixel, a period of 100 pixels. Here 0.2 is the sampling rate (one kept sample per 5 pixels).

Everyday versions. A wagon wheel that seems to spin backwards in a film. The swirling patterns on a TV when someone wears a fine striped shirt. Both are fast detail sampled too coarsely.

6.4 The fix: filter first, sample second

Once aliasing has happened, you cannot undo it: the fake slow wave looks identical to a real slow wave. So you must remove the too-fast detail **before** discarding samples. A low-pass filter, such as your Gaussian, does exactly that. That is why images are blurred before they are shrunk.

6.5 Why the test pattern is a chirp

A real photograph is usually already blurry from camera optics and JPEG compression, so it may not alias clearly. A **chirp** (also called a zone plate) is a synthetic pattern whose frequency *sweeps upwards* the further you go from the centre. It is guaranteed to contain every frequency from low to nearly the Nyquist limit.

$$\text{pattern} = \sin(2\pi \cdot (f_0 \cdot R + k \cdot R^2 / 2))$$

$f_0 = 10, \quad k = 250$ $R =$
distance from the centre, on a grid running from -0.5 to $+0.5$

The local frequency of this pattern is $f_0 + k \cdot R$ cycles per image width. With a 500-pixel width, divide by 500 for cycles per pixel:

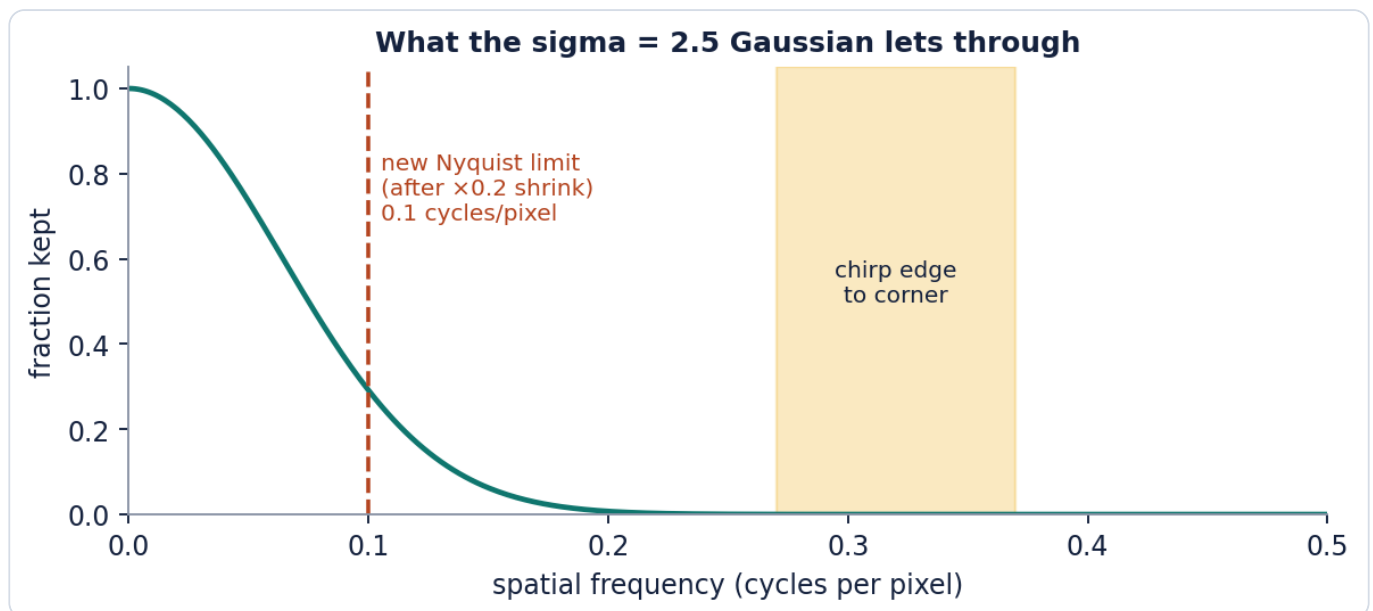
Where	R	Cycles per width	Cycles per pixel	vs new limit (0.1)
Centre	0	10	0.02	Safe
Ring where trouble starts	0.16	50	0.10	Exactly at the limit
Middle of an edge	0.50	135	0.27	Aliases
Corner	0.71	187	0.37	Aliases (still under 0.5, so the original can represent it)

So after a $\times 0.2$ shrink, everything outside about $R = 0.16$ (the dashed circle in the figure below) is above the limit and will alias if you do not filter first.

6.6 How well does the Gaussian remove those frequencies?

A Gaussian blur's effect on each frequency is itself a Gaussian: it keeps a fraction $\exp(-2\pi^2\sigma^2f^2)$ of frequency f . With $\sigma = 2.5$:

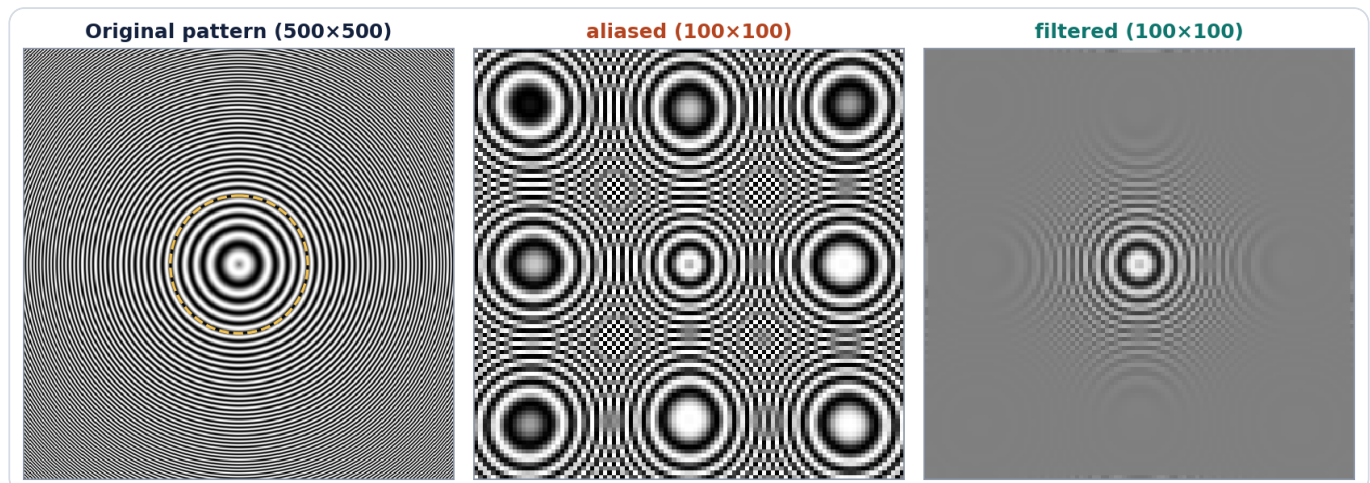
Frequency (cycles/pixel)	Fraction kept
0.05	73%
0.10 (the new limit)	29%
0.15	6%
0.27	0.01%
0.37	essentially 0



Frequencies well above the new limit are all but wiped out. Frequencies just above it are only partly reduced, so a little leftover remains. A Gaussian is a smooth low-pass, not a perfect cliff.

How the lab's $\sigma = 2.5$ was probably chosen. A common rule of thumb is $\sigma \approx 0.5 \times$ the shrink factor. Here the factor is $1 / 0.2 = 5$, so $\sigma \approx 2.5$. That is a rule of thumb, not a law. A larger σ removes more leakage but also blurs the parts you wanted to keep.

6.7 What you will see



A run of the lab's own pipeline ($\sigma = 2.5$, scale 0.2, order 0). The dashed circle marks $R = 0.16$.

- **Original:** concentric rings that get tighter towards the edges.
- **Aliased:** the centre looks right, but then *copies* of the ring pattern appear in a tiled grid, along with striped and checkerboard patches. None of these exist in the original.
- **Filtered:** clean rings near the centre that fade smoothly to grey. Faint ripples along the horizontal and vertical axes are most likely leftover leakage from the imperfect blur (Section 6.6). Nothing false has been invented.

The grey outer region is not a failure. Those frequencies cannot exist in a 100×100 image. The filtered path removes them honestly, and the naive path turns them into lies.

7. run_aliasing_demo

This is the one function in `evaluate_lab00B.py` that you complete. Right now its body is `raise NotImplementedError`.

7.1 What it does

It is a controlled experiment. You shrink one test pattern to 20% size in two ways. The *only* difference is whether a Gaussian blur was applied first. Everything else is identical: the same pattern, the same `SCALE_FACTOR`, the same `ORDER`. So any difference in the two results must come from the blur.

7.2 What you are given

Name	Value	Meaning
<code>pattern</code>	500×500 float array	The chirp, passed in as the function's argument
<code>SIGMA</code>	2.5	Blur width for the pre-filter
<code>SCALE_FACTOR</code>	0.2	Shrink to 20% (500 to 100)
<code>ORDER</code>	0	Nearest neighbour
<code>ConvolutionInterpolator</code>	your class	Already imported at the top of the file

`SIGMA`, `SCALE_FACTOR` and `ORDER` are module-level names defined near the top of the file. Use the names, not hard-coded numbers.

7.3 Pseudocode, then code

Pseudocode: read this first

```
run_aliasing_demo(pattern):
    1. ci = a new ConvolutionInterpolator

    # Path 1: shrink with no pre-filter
    2. aliased = ci.interpolate(pattern, scale_factor=SCALE_FACTOR, order=ORDER)

    # Path 2: blur first, then shrink
    3. kernel = ci.gaussian_kernel_2d(SIGMA)
    4. smoothed = ci.convolve(pattern, kernel) # still 500x500
    5. filtered = ci.interpolate(smoothed, scale_factor=SCALE_FACTOR, order=ORDER)

    6. return aliased, filtered # this order, as a pair
```

Expect: both results are 100x100. aliased shows fake patterns; filtered is clean.

Python

```
def run_aliasing_demo(pattern):
    ci = ConvolutionInterpolator()

    aliased = ci.interpolate(pattern, scale_factor=SCALE_FACTOR, order=ORDER)

    kernel = ci.gaussian_kernel_2d(SIGMA)
    smoothed = ci.convolve(pattern, kernel)
    filtered = ci.interpolate(smoothed, scale_factor=SCALE_FACTOR, order=ORDER)

    return aliased, filtered
```

Keep the function's docstring. Replace only the `raise NotImplementedError(...)` line with the body above.

7.4 Line by line

Line	What happens
<code>ci = ConvolutionInterpolator()</code>	Creates the helper object. It has no settings, so this just lets you call its methods.
<code>aliased = ci.interpolate(pattern, ...)</code>	Path 1. Nearest-neighbour keeps roughly every 5th pixel and discards the rest without looking at them.
<code>kernel = ci.gaussian_kernel_2d(SIGMA)</code>	Builds the 17×17 blur kernel.
<code>smoothed = ci.convolve(pattern, kernel)</code>	Blurs the full-size pattern. It stays 500×500, but the too-fast detail is now gone.
<code>filtered = ci.interpolate(smoothed, ...)</code>	Path 2's shrink. Same call as path 1, applied to the blurred image.
<code>return aliased, filtered</code>	Sends both 100×100 results back to <code>evaluate()</code> .

7.5 What "aliased" and "filtered" mean

Both are names for *outputs*, not for actions.

- `aliased` is the image from shrinking with no pre-filter, and it contains aliasing: the fake ring copies and moiré patches from Section 6.
- `filtered` is the image from blurring first and then shrinking. The name is a little misleading: it is both filtered *and* shrunk.

Neither is modified in place: `convolve` returns a *new* array, so `pattern` stays untouched, and both paths start from the same original.

7.6 What happens next in the script

`evaluate()` does the rest for you:

1. Builds the chirp with `_make_chirp_pattern()`.
2. Calls your `run_aliasing_demo(pattern)` and unpacks `aliased, filtered`.
3. Passes `pattern, aliased, filtered` to the provided `plot_aliasing_comparison()`. That helper uses Plotly `make_subplots` to build 3 side-by-side panels. Each is a `go.Heatmap` with a grey colour scale. `autorange="reversed"` flips the vertical axis because image rows run top to bottom.
4. Saves the figure with `save_html(...)` to `lab00B_results.html`.

You do not write any Plotly code. Just return the two arrays in the right order.

8. Running and checking your work

8.1 The commands

Goal	Command
Run the automated tests	<code>pytest test_lab00B.py</code>
Generate the figure	<code>python evaluate_lab00B.py</code> (the script's own header says to run it from the repository root as <code>python Assignments/lab00B/evaluate_lab00B.py</code>)
View the result	Open <code>lab00B_results.html</code> in a browser. In VS Code or Codespaces: right-click the file and choose <i>Open with Live Server</i> , or run <code>python -m http.server 5000</code> and open the forwarded port.

If `import helpers...` fails, you are almost certainly running from the wrong folder. The script adds the lab folder and its parent to `sys.path`, so run it as the header says.

8.2 A quick self-check before you run pytest

This is a small script that tests your methods against the numbers in this guide. It needs no helper files. Save it next to `lab00B.py` and run it.

Pseudocode: read this first

1. Make a `ConvolutionInterpolator`.
2. Build the kernel for sigma 2.5. Print its shape, its sum, and where its peak is.
3. Build the chirp pattern (same recipe as the lab).
4. Convolve with mode `same` / `valid` / `full` and print each output shape.
5. Make `aliased` and `filtered` as in `run_aliasing_demo`. Print their shapes.
6. Compare `aliased` against `skimage's rescale` with `anti_aliasing=False`. They must match.
7. Print the average absolute value in the outer part of each result.
`aliased` should be large; `filtered` should be close to 0.

Python

```
import numpy as np
from skimage.transform import rescale
from lab00B import ConvolutionInterpolator

ci = ConvolutionInterpolator()
k = ci.gaussian_kernel_2d(2.5)
print("kernel shape:", k.shape)
print("kernel sum  :", round(float(k.sum()), 6))
print("peak index  :", np.unravel_index(k.argmax(), k.shape))

x = np.arange(500) / 500 - 0.5
X, Y = np.meshgrid(x, x)
R = np.sqrt(X ** 2 + Y ** 2)
pattern = np.sin(2 * np.pi * (10 * R + 250 * R ** 2 / 2))

for m in ["same", "valid", "full"]:
    print(m, ci.convolve(pattern, k, mode=m).shape)

aliased = ci.interpolate(pattern, scale_factor=0.2, order=0)
filtered = ci.interpolate(ci.convolve(pattern, k), scale_factor=0.2, order=0)
print("result shapes:", aliased.shape, filtered.shape)

ref = rescale(pattern, 0.2, order=0, anti_aliasing=False, mode="reflect")
print("matches rescale (no anti-alias):", np.allclose(aliased, ref))

def outer_mean(a):
    h = a.shape[0]
    yy, xx = np.mgrid[:h, :h]
    r = np.hypot(yy - h / 2, xx - h / 2) / h
    return np.abs(a[r > 0.35]).mean()

print("outer mean |value|  aliased:", round(outer_mean(aliased), 3),
      " filtered:", round(outer_mean(filtered), 3))
```

What a correct solution prints:

Output

```
kernel shape: (17, 17)
kernel sum  : 1.0
peak index  : (np.int64(8), np.int64(8))
same (500, 500)
valid (484, 484)
full (516, 516)
result shapes: (100, 100) (100, 100)
matches rescale (no anti-alias): True
outer mean |value|  aliased: 0.636  filtered: 0.002
```

On older NumPy versions the peak line prints as `(8, 8)` instead of `(np.int64(8), np.int64(8))`. Both mean the same thing.

The `outer_mean` line is my own quick measure of "how much stripe is left in the outer region". The exact numbers matter less than the gap between them. If `filtered` is not tiny compared with `aliased`, your blur step is not working.

8.3 Before you submit Lab 00B

- No `raise NotImplementedError` is left in any of the four functions (three methods plus `run_aliasing_demo`).
- Every method has a `return`.
- The kernel for $\sigma = 2.5$ is 17×17 and sums to 1.
- Defaults are correct: `mode="same"`, `boundary="symm"`, `fillvalue=0`, `scale_factor=0.5`, `order=0`.
- `anti_aliasing=False` is in the `rescale` call.
- `run_aliasing_demo` returns `aliased`, `filtered` in that order and uses `SIGMA`, `SCALE_FACTOR`, `ORDER` rather than typed-in numbers.
- `pytest test_lab00B.py` passes, and the HTML shows a busy middle panel and a calm right panel.

9. Common mistakes

Mistake	What you will see	Fix
<code>int(3 * sigma)</code> instead of <code>ceil</code>	For $\sigma = 2.5$, $K = 15$ instead of 17. The size test fails.	<code>int(np.ceil(3 * sigma))</code>
<code>np.arange(-half, half)</code> (no <code>+ 1</code>)	An even-sized kernel (16 values) with no centre.	<code>np.arange(-half, half + 1)</code>
Forgetting to normalise	Sum is not 1 (for $\sigma = 1$ it is 0.99946). The sum test fails and brightness drifts.	<code>kernel / kernel.sum()</code>
<code>g * g</code> instead of <code>np.outer(g, g)</code>	A 1D result of shape (17,), not (17, 17).	Use <code>np.outer</code>
Passing a kernel size as sigma	A giant kernel. $\sigma = 17$ would give $K = 103$.	Sigma is the width parameter. K is derived from it.
Leaving <code>anti_aliasing</code> at its default	<code>aliased</code> looks smooth and clean. The "not anti-aliased" test fails.	Pass <code>anti_aliasing=False</code>
Ignoring kwargs and hard-coding <code>mode="same"</code>	The <code>mode="valid"</code> test fails.	Read every setting with <code>kwargs.get(name, default)</code>
Wrong default (for example <code>boundary="fill"</code>)	Darkened image edges. Boundary test fails.	Defaults: <code>"same"</code> , <code>"symm"</code> , <code>0</code>
Blurring <i>after</i> shrinking	<code>filtered</code> looks as aliased as <code>aliased</code> .	Convolve first, then interpolate
Using <code>cv2.GaussianBlur</code> in Lab 00B	Values differ from the reference kernel. It is not the point of the lab.	Use your own kernel and <code>convolve2d</code>
Typing <code>2.5</code> or <code>0.2</code> into <code>run_aliasing_demo</code>	Works today, breaks if a constant changes.	Use <code>SIGMA</code> , <code>SCALE_FACTOR</code> , <code>ORDER</code>
Returning <code>filtered, aliased</code>	The panels are silently mislabelled.	Return <code>aliased, filtered</code>
No <code>return</code> , or <code>raise NotImplementedError</code> still there	<code>None</code> comes back, or the program stops with that error.	Replace the <code>raise</code> line and return your result
Lab 00: casting to <code>uint8</code> before clipping	Bright pixels wrap to dark ones.	Clip first, cast second
Lab 00: an even <code>ksize</code> passed to OpenCV	An OpenCV error.	Add 1 when even

10. Glossary

Aliasing

False, slower patterns that appear when detail is sampled too coarsely. Fast detail "masquerades" as slow detail.

Anti-aliasing

Anything that prevents aliasing, usually a low-pass filter applied before shrinking.

Array (ndarray)

NumPy's grid of same-typed numbers. An image is one.

Bicubic (order 3)

Interpolation that fits a smooth curve through 16 nearby pixels.

Bilinear (order 1)

Interpolation that averages the 4 nearest pixels, weighted by distance.

Boundary

The rule for what a convolution assumes lies beyond the image edge: `symm` , `fill` or `wrap` .

Channel

One colour layer of an image. A colour image has 3.

Chirp (zone plate)

A test pattern whose frequency rises with distance from the centre.

Class

A blueprint bundling data and methods.

Clip

Cap values into a range, e.g. `np.clip(x, 0, 255)` .

Convolution

Sliding a flipped kernel over an image, multiplying and summing at every position.

Correlation

Like convolution but without flipping the kernel.

Cycles per pixel

The unit of spatial frequency. Its maximum is 0.5.

Downsampling

Making an image smaller (`scale_factor` below 1).

dtype

The number type stored in an array, e.g. `uint8` or `float64` .

Energy

The sum of squared magnitudes of a spectrum over some region.

FFT

Fast Fourier Transform. It reports how much of each frequency an image contains.

Filter

An operation that changes an image's frequency content.

Fillvalue

The constant used to pad the edges when `boundary="fill"`.

Gaussian

The bell-shaped curve $\exp(-x^2/2\sigma^2)$. Used as a blur weight.

High-pass / low-pass

A filter that keeps fast changes / slow changes.

HF / LF

High-frequency / low-frequency content.

Instance

A specific object made from a class, e.g. `ci`.

Interpolation

Estimating pixel values at new positions when resizing.

Kernel

A small grid of weights used to filter an image.

Keyword argument

An argument passed by name, e.g. `mode="valid"`.

`**kwargs`

A parameter that collects all keyword arguments into a dictionary.

Magnitude spectrum

The picture of "how much of each frequency" produced by the FFT.

Method

A function that belongs to a class.

Mode (`convolve`)

Chooses output size: `same`, `valid` or `full`.

Mode (`rescale`)

How the edges are padded during interpolation. Here `'reflect'`.

Moiré

The swirling patterns caused by aliasing of fine repeating detail.

Nearest neighbour (order 0)

Interpolation that copies the closest pixel, with no averaging.

Normalise

Rescale so that something adds up to 1 (here, the kernel weights).

Nyquist limit

The highest frequency a sampling grid can represent: 2 samples per cycle, or 0.5 cycles per pixel.

Order

Which interpolation method to use: 0, 1 or 3.

Outer product

A grid whose entry $[i, j]$ equals $a[i] \times b[j]$. `np.outer`.

Padding

Extra pixels added around the image so a kernel can be applied at the border.

Positional argument

An argument matched by its position in the call.

Pre-filter

A filter applied *before* another operation, here before shrinking.

Resampling

Reading an image at a new set of positions, i.e. changing its pixel grid.

Sample

One recorded value of a signal, i.e. one pixel.

Scale factor

New size divided by old size.

`self`

The object a method is being called on. Python passes it in automatically.

Separability

A 2D Gaussian equals a 1D Gaussian in x times a 1D Gaussian in y.

Sigma (σ)

The Gaussian's standard deviation. Bigger means a wider bell and a stronger blur.

Spatial frequency

How fast brightness changes across an image.

Symmetric kernel

One that reads the same forwards and backwards. Flipping it changes nothing.

`uint8`

Unsigned 8-bit integer: whole numbers 0 to 255. It wraps around outside that range.

Unsharp mask

Sharpening by adding a scaled copy of (original – blurred) back onto the original.

Upsampling

Making an image larger (`scale_factor` above 1).

valid / same / full

Convolution output sizes: only full overlap / same as input / any overlap.

11. Cheat sheet

The idea in three lines

Fine detail is high frequency. A Gaussian blur removes it. Blur **before** you shrink, or the leftover detail turns into fake patterns (aliasing).

Lab 00B methods

Method	Pseudocode
<code>gaussian_kernel_2d(sigma)</code>	<code>half = int(ceil(3σ)); x = arange(-half, half+1); g = 1/(σ√2π)·exp(-x²/2σ²); K = outer(g, g); K / K.sum()</code>
<code>convolve(image, kernel, **kw)</code>	<code>convolve2d(image, kernel, mode=kw("mode", "same"), boundary=kw("boundary", "symm"), fillvalue=kw("fillvalue", 0))</code>
<code>interpolate(image, **kw)</code>	<code>rescale(image, kw("scale_factor", 0.5), order=kw("order", 0), anti_aliasing=False, mode='reflect')</code>
<code>run_aliasing_demo(pattern)</code>	<code>aliased = interpolate(pattern); filtered = interpolate(convolve(pattern, kernel(SIGMA))); return aliased, filtered</code>

In this table, `kw(name, default)` is short for `kwargs.get(name, default)`.

Lab 00 methods

<code>add_blur</code>	<code>GaussianBlur(image, (ksize, ksize), 0)</code> , with ksize odd, default 15
<code>add_sharpen</code>	<code>addWeighted(image, 1+strength, blurred, -strength, 0)</code> , then clip 0 to 255, then <code>uint8</code> . Defaults: ksize 15, strength 1.5

Numbers to remember

Kernel size	$K = 2 \cdot \text{ceil}(3\sigma) + 1$. For $\sigma = 2.5$, $K = 17$.
Output sizes (500×500 image, $K = 17$)	same 500 · valid 484 ($H-K+1$) · full 516 ($H+K-1$)
Shrink ×0.2	500×500 becomes 100×100
Nyquist limit	0.5 cycles/pixel, and 0.1 cycles/pixel after a ×0.2 shrink
Chirp local frequency	$f_0 + k \cdot R$ cycles per width (divide by 500 for per pixel). Trouble starts at $R \approx 0.16$.
Unsharp mask	<code>original + strength × (original – blurred)</code>
Gaussian frequency response	$\exp(-2\pi^2\sigma^2f^2)$

Defaults

<code>convolve</code>	<code>mode="same", boundary="symm", fillvalue=0</code>
<code>interpolate</code>	<code>scale_factor=0.5, order=0</code> (<code>anti_aliasing</code> always <code>False</code> , <code>mode='reflect'</code>)
Demo constants	<code>SIGMA=2.5, SCALE_FACTOR=0.2, ORDER=0</code>

Do not forget

- `ceil`, not `int`, and `half + 1` in `arange`.
- Normalise the kernel. Use `np.outer`, not `*`.
- `anti_aliasing=False`.
- Filter first, shrink second.
- Return `aliased`, `filtered`, in that order.